

Notion 프로젝트 섹션 원고

노션 **프로젝트** 데이터베이스에 붙여넣기 좋은 형태로 정리한 원고입니다.
각 프로젝트 페이지의 첫 화면에는 **한 줄 요약**, **문제**, **내 역할**, **결과**가 먼저 보이도록 구성했습니다.

프로젝트 카드 요약

프로젝트	유형	기간	역할	한 줄 요약	핵심 지표
WorkFlow Agent (듀 듀)	Team / AI Agent	2026.02 - 2026.04	AI Engineer	사내 규정 판단이 수동 검색 10~15분에 의존하던 문제를 LangGraph 멀티 Agent와 4중 Guardrail로 해결해 판단 정확도를 37%에서 85%로 개선	Intent F1 97.88%, JSON 유효성 70%→97%, 판단 정확도 37%→85%
PyMate	Team / RAG Tutor	2026.01 - 2026.02	Backend & Frontend	강의 자료 기반 답변의 검색 품질 병목을 RAGAS로 분리 측정하고 임베딩·리랭킹 전략을 바꿔 Context Precision을 0.83에서 0.97로 개선	Precision 0.83→0.97, Recall 0.70→0.79, 퀴즈 유형 1→3개
CryptoVol Dashboard	Personal / Fullstack	2026.03	Fullstack Developer	Jupyter 분석 코드에 머물던 GARCH 변동성 모델을 FastAPI + React 실시간 대시보드로 전환해 5개 모델 비교·예측·검증을 서비스화	5개 GARCH 모델, 14개 API, 5분 TTL 캐싱, Monte Carlo 10,000회
Seoul Culture Map	Personal / Data Service	2026.03	Fullstack Developer	PDF 보고서에 갇힌 서울 문화시설 분석을 인터랙티브 지도와 RAG 추천 서비스로 확장해 2,500개 이상 시설 탐색을 가능하게 함	2,500+ 시설, 11개 API, 19개 지하철 노선, AI 요청 비용 85% 절감
암호화폐 변동성 비교 분석	Team / Statistical Research	2024	Data Analyst	비트코인 변동성 예측에서 외생변수의 효과를 검증하기 위해 2,129거래일 데이터를 5개 GARCH 계열 모델로 비교	2,129거래일, 5개 모델, BTC-FNG $r=0.72$, HAR-TGARCH-X 최적
서울시 문화·여가시설 분석	Team / Statistical Research	2023.09 - 2023.10	Data Analyst	서울 25개 자치구 문화시설 분포 불균형을 6개 카테고리 공공데이터로 정량화하고 군집별 관광 전략을 제안	25개 자치구, 6개 카테고리, PCA 설명력 91.3%, 학술제 2위

프로젝트	유형	기간	역할	한 줄 요약	핵심 지표
SubFlow	Personal	2026.03	Fullstack Developer	흠어진 구독 지출을 한 화면에서 관리하기 위해 Web/Mobile/FastAPI를 공유하는 멀티 클라이언트 구독 관리 플랫폼을 설계	Web + Mobile, 8개 DB 테이블, 6종 API 라우터, 30+ 서비스 카탈로그
	/	-			
	Fullstack	2026.04			

1. WorkFlow Agent (듀듀)

한 줄 요약

사내 규정 판단이 수동 검색 10~15분에 의존하던 문제를 LangGraph 멀티 Agent와 4중 Guardrail로 해결해 판단 정확도를 37%에서 85%로 개선한 프라이빗 업무 자동화 시스템입니다.

프로젝트 정보

- 유형: Team Project / SK Networks AI Camp Final
- 기간: 2026.02 - 2026.04
- 팀 규모: 5명
- 내 역할: AI Engineer
- 담당 범위: Judgment Agent, RAG 파이프라인, LoRA 데이터 품질 실험, vLLM 서빙 안정화, 4중 Guardrail, 5-factor Confidence 보정
- 기술: LangGraph, Kanana-1.5-8B, LoRA, vLLM, FastAPI, Qdrant, BM25, RRF, Reranker, AWS, Docker

Problem

사내 규정 확인은 여러 문서를 사람이 직접 검색해야 해서 1건당 10~15분이 걸렸고, 외부 GPT API를 그대로 쓰기에는 사내 데이터 보안과 비용 부담이 있었습니다. 또한 초기 sLLM은 confidence를 0.9 이상으로 출력해도 실제 판단 정확도는 37% 수준에 머물러, 모델 답변을 그대로 서비스에 노출하기 어려웠습니다.

Approach

하나의 거대한 챗봇으로 모든 업무를 처리하는 대신, Intent 분류 이후 Judgment / Document / Schedule / Planner Agent로 책임을 분리했습니다. 특히 제가 맡은 Judgment Agent는 규정 문서를 HyDE + BM25 + Vector + RRF + Reranker로 검색하고, sLLM 판단 결과를 4중 Guardrail과 5-factor Confidence로 다시 검증하는 구조로 설계했습니다.

Results

- 규정 판단 정확도: 37% → 85%
- JSON 출력 유효성: 70% → 97%
- Intent 분류: Test F1 97.88%, Adversarial F1 87.58%, 추론 7.9ms
- confidence 보정: 과신된 0.92 평균을 0.78 수준으로 보정
- 수동 규정 검색: 10~15분 → 자연어 질의 10초 이내 응답 구조

My Contributions

- sLLM 판단 결과를 그대로 믿을 수 없는 문제 → 4중 Guardrail과 5-factor Confidence 보정 설계 → 판단 정확도 37%에서 85%로 개선
- 단일 벡터 검색이 규정 교차 판단을 놓치는 문제 → HyDE + BM25 + Vector + RRF + Reranker 9단계 검색 파이프라인 설계 → 복수 규정 근거를 함께 검색
- 데이터 양을 늘렸는데 성능이 떨어지는 문제 → LoRA 데이터 v1~v3를 반복 실험하며 품질 기준 재정의 → v2 -3.2%p 하락 후 v3 +2.0%p 회복
- GPT API 의존으로 비용·보안 리스크가 있던 구조 → vLLM 기반 Kanana-1.5-8B 프라이빗 서빙으로 전환 → 외부 API 의존 제거

Retrospective

이 프로젝트에서 가장 크게 배운 것은 “LLM이 답을 냈는가”보다 “그 답을 서비스가 믿어도 되는가”가 더 중요하다는 점입니다. 모델 성능을 높이는 것만으로는 부족했고, 검색 근거·출력 형식·판단 범위·confidence를 서버에서 다시 검증하는 구조가 있어야 실제 업무 자동화가 가능했습니다.

2. PyMate

한 줄 요약

부트캠프 수강생이 강의 외 시간에 질문을 해결하기 어려운 문제를 RAG 기반 AI 튜터로 해결하고, 검색 품질 병목을 RAGAS로 측정해 Context Precision을 0.83에서 0.97로 개선했습니다.

프로젝트 정보

- 유형: Team Project / SKN21 3차 + 4차
- 기간: 2026.01 - 2026.02
- 팀 규모: 6명
- 내 역할: Backend & Frontend
- 담당 범위: RAG 품질 개선, Quiz API, 오답 재학습, 에러 미들웨어, Flask → Django 전환, AWS 배포
- 기술: Django, DRF, Flask, LangChain, LangGraph, Qdrant, RAGAS, PostgreSQL, AWS EC2, Nginx, Gunicorn

Problem

부트캠프 수강생은 강의 외 시간에 질문을 즉시 해결하기 어려웠고, 일반 LLM은 실제 강의 자료와 다른 답변을 생성할 위험이 있었습니다. 초기 MVP에서는 답변 품질 문제가 LLM 자체의 한계처럼 보였지만, RAGAS로 나눠 측정해보니 병목은 생성 모델보다 검색 단계에 있었습니다.

Approach

Flask MVP에서 시작한 RAG 챗봇을 Django 기반 프로덕션 구조로 전환했습니다. 검색 품질은 RAGAS 지표로 Context Precision / Recall을 분리 측정했고, 임베딩 차원과 리랭커를 바꾸며 실제 검색 결과를 비교했습니다. 기능 측면에서는 챗봇에 머물지 않고 퀴즈, 코드 리뷰, 스튜디오, 북마크까지 학습 흐름을 확장했습니다.

Results

- Context Precision: 0.8333 → 0.9758 (+14.4%p)
- Context Recall: 0.7044 → 0.7944 (+12.7%p)
- 임베딩: 768D → 3,072D

- 퀴즈 유형: 1개 → 객관식/OX/단답형 3개
- 응답 방식: 일반 응답 → SSE 실시간 스트리밍
- 배포: 로컬 MVP → AWS EC2 + Docker + Nginx + Gunicorn

My Contributions

- RAG 답변 품질 저하 원인이 불명확한 문제 → RAGAS로 검색과 생성을 분리 평가 → 병목을 LLM이 아닌 embedding 검색 품질로 재정의
- 단일 퀴즈 형식만 지원해 학습 검증이 제한되는 문제 → DRF 기반 3종 Quiz API 설계 → 퀴즈 유형 1개에서 3개로 확장
- 틀린 문제를 다시 찾기 어려운 문제 → QuizBookmark와 Qdrant를 연동한 오답 재학습 흐름 설계 → 마이페이지에서 즉시 재학습 가능
- Flask MVP의 인증·ORM·정적파일 구성이 모두 수동이던 문제 → Django로 마이그레이션 → 사용자 인증, 관리자 기능, 배포 구조 확보
- AWS 배포 시 502 에러가 발생하는 문제 → Nginx + Gunicorn 경로와 서비스 설정 디버깅 → 프로덕션 배포 완료

Retrospective

처음에는 "LLM이 더 좋아야 한다"고 생각했지만, 실제로는 검색 품질이 답변 품질을 결정하고 있었습니다. 이 경험 이후에는 AI 기능을 만들 때 모델 교체보다 먼저 검색·컨텍스트·평가 지표를 분리해 보는 습관이 생겼습니다.

3. CryptoVol Dashboard

한 줄 요약

Jupyter에서만 실행되던 GARCH 변동성 분석을 FastAPI + React 실시간 대시보드로 전환해 5개 모델 비교, 매매 시그널, 포트폴리오 리스크 분석까지 서비스화했습니다.

프로젝트 정보

- 유형: Personal Project / 팀 분석 프로젝트 개인 확장
- 기간: 2026.03
- 내 역할: Fullstack Developer
- 담당 범위: FastAPI API 설계, GARCH 모델 서빙, WebSocket 릴레이, 캐싱 전략, React 대시보드, 배포
- 기술: FastAPI, React, Recharts, GARCH, Binance WebSocket, CoinGecko API, OpenAI API, Docker, Render, Vercel

Problem

기존 암호화폐 변동성 분석은 Jupyter Notebook에서 수동 실행해야 했기 때문에, 실시간 가격 변화에 따라 모델을 비교하거나 투자 판단에 활용하기 어려웠습니다. GARCH 모델 적합은 요청마다 수백 ms 이상 걸렸고, 한 모델 실패가 전체 응답 실패로 번질 위험도 있었습니다.

Approach

분석 코드를 서비스 코드로 전환하며, 모델 계산과 실시간 가격 수신을 분리했습니다. Binance WebSocket은 백엔드에서 릴레이하고, CoinGecko와 GARCH 결과는 API로 제공했습니다. 비용이 큰 모델 적합 결과는 5분 TTL

캐싱으로 제어하고, 모델별 try-except 격리로 하나의 실패가 전체 대시보드를 멈추지 않도록 했습니다.

Results

- GARCH 계열 5개 모델 실시간 비교: GARCH, TGARCH, HAR-GARCH, HAR-TGARCH, HAR-TGARCH-X
- API 구성: 13 REST + 1 WebSocket
- 데이터 범위: 2,129 거래일 기반 모델 비교
- 포트폴리오 리스크: Monte Carlo 10,000 시나리오
- 캐싱: 5분 TTL로 반복 계산 비용 제어
- 검증: pytest 28개 + GitHub Actions CI 구성

My Contributions

- Notebook 분석 결과가 서비스에서 재사용되지 못하는 문제 → GARCH 모델 적합 로직을 FastAPI API로 분리 → 웹 대시보드에서 모델 비교 가능
- 실시간 가격 API를 프론트에서 직접 호출할 때 CORS와 보안 문제가 생기는 문제 → Binance WebSocket 백엔드 릴레이 설계 → 프론트는 안정적인 단일 WS만 구독
- GARCH 적합 비용이 요청마다 반복되는 문제 → 5분 TTL 인메모리 캐싱 적용 → 반복 요청 계산 비용 감소
- 하나의 모델 실패가 전체 응답 실패로 이어지는 문제 → 모델별 에러 격리와 fallback 응답 설계 → 5개 모델 중 일부 실패에도 대시보드 유지
- 1,000회 시뮬레이션에서 99% VaR 꼬리 분포가 불안정한 문제 → Monte Carlo 10,000회로 확장 → 리스크 지표 안정화

Retrospective

분석 코드는 “정답을 계산하는 코드”에 가깝지만, 서비스 코드는 “실패해도 사용자 경험을 유지하는 코드”여야 했습니다. 이 프로젝트를 통해 캐싱, 에러 격리, API 응답 설계가 분석 모델만큼 중요하다는 것을 배웠습니다.

4. Seoul Culture Map

한 줄 요약

서울 문화시설 분석 결과가 PDF 보고서에 머물러 활용성이 낮던 문제를 2,500개 이상 시설을 탐색할 수 있는 인터랙티브 지도와 AI 추천 서비스로 확장했습니다.

프로젝트 정보

- 유형: Personal Project / 학술제 분석 프로젝트 개인 확장
- 기간: 2026.03
- 내 역할: Fullstack Developer
- 담당 범위: FastAPI API 11개, Leaflet 지도, K-means 군집분석, 지하철 접근성 분석, LangGraph RAG 추천, SSE 스트리밍
- 기술: FastAPI, React, Leaflet, scikit-learn, ChromaDB, LangGraph, OpenAI API, SQLite/PostgreSQL

Problem

기존 서울 문화시설 분석은 발표 자료와 PDF에 정리되어 있었지만, 사용자가 직접 지역·시설·교통 접근성을 탐색하거나 “종로 근처 조용한 박물관” 같은 자연어 질문을 던질 수는 없었습니다. 분석 결과는 있었지만 서비스로 연결되지 않아 실제 사용성이 낮았습니다.

Approach

R 분석 결과를 웹 서비스 데이터로 재가공하고, Leaflet 지도 위에 시설·군집·권역·지하철 접근성을 겹쳐 볼 수 있게 만들었습니다. 자연어 추천은 LangGraph 3-node Agent(Intent → Retrieve → Generate)로 분리하고, ChromaDB RAG와 SSE 스트리밍을 적용했습니다.

Results

- 시설 데이터: 2,500개 이상
- 지도 필터: 7개 카테고리, 서울 5대 권역, 19개 지하철 노선
- API: 11개 엔드포인트
- 분석 모드: 군집분석, 권역별, 카테고리 밀도, 지하철 접근성 4개 모드
- AI 추천: 요청당 약 \$0.003 수준으로 비용 85% 절감

My Contributions

- PDF 보고서로는 시설 분포를 직접 탐색하기 어려운 문제 → Leaflet 기반 인터랙티브 지도 구현 → 2,500개 이상 시설을 카테고리별로 탐색 가능
- 문화 접근성 판단에 교통 데이터가 빠져 있는 문제 → 19개 지하철 노선 필터와 반경 1.5km 역 수 분석 추가 → 시설 접근성 비교 가능
- 키워드 검색만으로 자연어 요구를 처리하기 어려운 문제 → LangGraph Agent + ChromaDB RAG 설계 → “지역+분위기+시설 유형” 복합 질의 대응
- 정적 분석 결과가 개인화되지 않는 문제 → 즐겨찾기, 코스 추천, AI 추천 흐름 추가 → 사용자가 직접 문화 코스를 구성 가능

Retrospective

분석 프로젝트를 서비스로 확장하면서, “좋은 분석”과 “사용자가 다시 열어보는 서비스”는 다르다는 것을 배웠습니다. 분석 지표를 지도 위의 행동 가능한 인터페이스로 바꾸는 과정에서 데이터 시각화와 UX 설계의 연결을 체감했습니다.

5. 암호화폐 변동성 비교 및 분석

한 줄 요약

비트코인 변동성 예측에서 외생변수의 효과를 검증하기 위해 2,129거래일 데이터를 5개 GARCH 계열 모델로 비교하고 HAR-TGARCH-X의 우수성을 확인했습니다.

프로젝트 정보

- 유형: Team Project / Statistical Research
- 기간: 2024
- 내 역할: Data Analyst
- 담당 범위: 데이터 전처리, 사전 검정, GARCH 모델 비교, 외생변수 효과 분석, 발표
- 기술: Python, pandas, numpy, arch, statsmodels, scipy, matplotlib

Problem

비트코인은 급락 시 변동성이 비대칭적으로 커지는 특성이 있는데, 단순 GARCH 모델만으로는 하락 충격과 시장 심리 변수를 충분히 반영하기 어려웠습니다. 따라서 변동성 예측에 HAR 구조, TGARCH 비대칭 효과, 거래량과 FNG 외생변수가 실제로 기여하는지 검증할 필요가 있었습니다.

Approach

2018년 2월부터 2023년 11월까지 약 2,129거래일 데이터를 수집하고, 로그수익률 변환 후 ADF와 ARCH-LM 검정으로 모델 적용 근거를 확인했습니다. 이후 GARCH, TGARCH, HAR-GARCH, HAR-TGARCH, HAR-TGARCH-X 5개 모델을 1-step ahead 방식으로 비교했습니다.

Results

- 분석 기간: 2018.02 - 2023.11, 약 2,129 거래일
- 모델 수: 5개 GARCH 계열 모델 비교
- BTC-FNG 상관: $r=0.72$, $p<0.001$
- TGARCH 레버리지 효과: $\gamma=0.0990$
- 최적 모델: HAR-TGARCH-X

My Contributions

- 가격 데이터가 비정상성을 가져 바로 모델링하기 어려운 문제 → 로그수익률 변환과 ADF 검정 수행 → 정상성 기반 분석 데이터 확보
- GARCH 적용 근거가 필요했던 문제 → ARCH-LM 검정으로 이분산성 확인 → 조건부 변동성 모델 적용 타당성 확보
- 단일 모델로는 하락 충격을 반영하기 어려운 문제 → TGARCH와 HAR 구조를 결합 비교 → 비대칭 변동성 반영 모델 검증
- 외생변수 효과가 직관에 머무르던 문제 → Volume + FNG를 HAR-TGARCH-X에 주입 → 시장 심리 변수의 예측 기여 확인

Retrospective

이 프로젝트는 이후 CryptoVol Dashboard로 확장되는 출발점이었습니다. 연구 단계에서는 모델의 설명력과 검정이 중요했다면, 서비스 단계에서는 같은 모델을 어떻게 빠르고 안정적으로 제공할지가 새 과제가 된다는 것을 알게 되었습니다.

6. 서울시 문화·여가시설 현황 분석

한 줄 요약

서울 25개 자치구의 문화시설 분포 불균형을 6개 카테고리 공공데이터로 정량화하고, PCA와 군집분석으로 관광 목적에 맞는 지역 그룹을 제안한 학술제 프로젝트입니다.

프로젝트 정보

- 유형: Team Project / Statistical Research
- 기간: 2023.09 - 2023.10
- 내 역할: Data Analyst
- 담당 범위: 공공데이터 전처리, 시설별 집계, 군집분석, PCA, 시각화, 발표
- 기술: R, dplyr, public data, PCA, clustering

- 성과: 학술제 2위

Problem

서울의 문화·여가시설은 자치구별로 밀집도가 크게 다르지만, 시설 유형별로 어느 지역이 어떤 강점을 갖는지 한눈에 비교하기 어려웠습니다. 공공데이터도 법정동, 행정동, 시군구 단위가 섞여 있어 바로 분석하기 어려웠습니다.

Approach

공연시설, 박물관/유적지, 공원, 방탈출, 영화관, 전통사찰 등 6개 카테고리 데이터를 수집해 서울시 기준으로 필터링하고, 행정동-시군구 매핑으로 지역 단위를 통일했습니다. 이후 25개 자치구를 기준으로 시설 분포를 집계하고, PCA와 군집분석으로 지역별 특성을 분류했습니다.

Results

- 분석 대상: 서울 25개 자치구
- 시설 카테고리: 6개
- PCA 설명력: 91.3%
- 군집 수: 최적 $k=3$
- 성과: 학술제 2위

My Contributions

- 데이터마다 법정동·행정동 단위가 섞인 문제 → 행정동-시군구 매핑 테이블로 단위 통일 → 25개 자치구 비교 가능
- 폐업·휴업 시설이 포함되어 현황 분석을 왜곡하는 문제 → 영업 상태와 카테고리 필터링 적용 → 현재 운영 시설 중심 데이터셋 생성
- 단순 시설 수 비교로는 지역 특성을 설명하기 어려운 문제 → PCA와 군집분석 적용 → 관광 목적별 지역 그룹 도출
- 분석 결과가 발표 자료에 머무르는 문제 → 이후 Seoul Culture Map 개인 프로젝트로 확장 → 인터랙티브 지도 서비스의 기반 데이터로 재사용

Retrospective

이 프로젝트를 통해 데이터 전처리의 대부분은 모델링 이전의 기준 통일에서 시작된다는 것을 배웠습니다. 같은 서울시 데이터라도 지역 단위와 시설 상태가 다르면 분석 결과가 달라지기 때문에, 분석 설계의 첫 단계는 “무엇을 같은 기준으로 비교할 것인가”를 정하는 일이라는 판단 기준을 얻었습니다.

7. SubFlow

한 줄 요약

흩어진 구독 서비스를 한 화면에서 관리하기 위해 React Web, React Native Mobile, FastAPI Backend를 공유하는 멀티 클라이언트 구독 관리 플랫폼을 설계했습니다.

프로젝트 정보

- 유형: Personal / Side Project

- 기간: 2026.03 - 2026.04
- 내 역할: Fullstack Developer
- 담당 범위: React Web, React Native Mobile, FastAPI Backend, PostgreSQL schema, JWT 인증, 지출 분석
- 기술: React 19, React Native, Expo, FastAPI, SQLAlchemy 2.0, PostgreSQL, Alembic, Zustand, Recharts

Problem

Netflix, Spotify, YouTube Premium, ChatGPT Plus처럼 구독 서비스가 늘어나면서 사용자가 월별·연간 지출을 정확히 파악하기 어려웠습니다. 특히 중복 구독, 무료체험 만료, 외화 결제 환율 변동은 사용자가 직접 기억하지 않으면 놓치기 쉬운 지출 리스크였습니다.

Approach

Web과 Mobile이 같은 FastAPI 백엔드를 공유하는 구조로 설계했습니다. 구독 상태, 결제 주기, 카테고리, 환율, 알림 데이터를 PostgreSQL에 정규화하고, 프론트에서는 대시보드·분석·캘린더·타임라인 뷰로 사용자가 반복적으로 확인해야 하는 정보를 빠르게 볼 수 있게 구성했습니다.

Results

- 클라이언트: React Web + React Native Mobile
- 백엔드: FastAPI 단일 API
- DB: PostgreSQL 16, 8개 테이블
- API: `/api/v1` 기준 6종 라우터
- 서비스 카탈로그: 30개 이상
- 기능: 지출 분석, 중복 구독 감지, 예산 경고, 환율 추적, 무료체험 만료 D-day

My Contributions

- 구독 지출이 여러 앱에 흩어져 총액을 알기 어려운 문제 → 월/연간 지출 대시보드 설계 → 반복 결제 규모를 즉시 확인 가능
- 음악·영상 등 중복 구독을 놓치는 문제 → 카테고리 기반 중복 감지 로직 설계 → 절약 제안 제공
- 외화 구독의 실제 원화 지출이 변동되는 문제 → Frankfurter API 환율 연동 → KRW 자동 변환과 임계치 알림 가능
- Web과 Mobile을 따로 만들면 API 중복이 커지는 문제 → 단일 FastAPI 백엔드와 JWT 인터셉터 구조 설계 → 멀티 클라이언트 확장 기반 확보

Retrospective

SubFlow는 기능을 많이 넣는 것보다 반복 사용자가 매달 확인할 정보를 얼마나 빨리 이해하게 할지가 핵심이었습니다. 그래서 대시보드는 화려한 설명보다 총액, 다음 결제일, 예산 초과, 중복 구독처럼 바로 행동으로 이어지는 지표를 우선 배치했습니다.